Real-Time Incident Management Dashboard

Final Report for the Distributed Systems Course 2025/2026

Alessandro Martini
alessandro.martini10@studio.unibo.it

October 15, 2026

This project presents a distributed real-time incident management dashboard designed for collaborative ticket resolution in operational environments. The system enables multiple operators to simultaneously interact with a shared pool of incidents through a web-based dashboard. The architecture follows a three-tier pattern: a Vue.js 3 single-page application serves as the presentation layer, an Express.js API with Socket.IO handles real-time communication and business logic, while a MongoDB Replica Set ensures persistent strongly-consistent state and Redis provides distributed coordination. Key distributed systems patterns implemented include: (i) Redis-based distributed locking and Lua scripts for check-then-act operations, preventing race conditions during ticket assignment; (ii) optimistic locking via document versioning in MongoDB to avoid lost updates during concurrent modifications; (iii) leader election using Redis TTL keys for exclusive execution of background tasks (automatic escalation of stale incidents); (iv) event-driven synchronization of all connected clients without polling, combining MongoDB Change Streams for incident updates with Redis Pub/Sub (via the Socket.IO adapter) for lock and presence events. The entire system is containerized and orchestrated via Kubernetes (validated via Minikube), featuring a 3-node MongoDB StatefulSet for data persistence and failover, Horizontal Pod Autoscaler for the API tier (3–10 replicas), and an Ingress with session affinity.

Disclaimer

During the preparation of this work, the author used an LLM to help design the acceptance test procedures

After using this tool/service, the author reviewed and edited the content as needed and take full responsibility for the content of the final report.

1 Concept

1.1 Product Type

The project consists of a web application with a real-time graphical user interface (GUI). Specifically, it is a Single-Page Application (SPA) built with Vue.js 3, served as a static web application and interacting with a distributed backend via HTTP and WebSocket protocols. The frontend presents a Kanban-style dashboard that allows multiple operators to visualize, create, claim, and resolve incidents in real time.

1.2 Use Case Description

1.2.1 What the software does

The system implements a **real-time incident management board** (similar to a ticketing or ITSM system) with the following core capabilities:

- **Incident registration:** operators can create new incidents with a title and initial status `open`
- **Ticket claiming:** an operator can *lock* an incident for exclusive processing, preventing other operators from simultaneously working on the same ticket (mutual exclusion)
- **Ticket resolution:** the claiming operator can mark an incident as `closed`, releasing the lock and notifying all connected clients
- **Automatic escalation:** a background task periodically scans for incidents that have remained `open` for more than 5 minutes and automatically escalates them to `escalated` status, signaling SLA breach
- **Real-time synchronization:** all connected clients receive live updates whenever an incident is created, modified, locked, unlocked, or escalated, without manual page refresh
- **User presence tracking:** the system displays the list of currently connected operators

1.3 Why Distribution is Needed

1.3.1 Fault Tolerance

The system must remain operational even if individual components fail. With a single-node deployment, the failure of the API server or the database would render the entire ticketing system unusable—a critical risk in operational environments where incident tracking cannot afford downtime. The Kubernetes architecture addresses this by:

- Running **3 API replicas** minimum, so the loss of one pod does not interrupt service

- Deploying MongoDB as a **3-node Replica Set**, which tolerates the failure of one node, electing a new primary if needed
- Using Redis for shared state, so any API pod can take over coordination tasks

Redis itself, however, runs as a single instance and is the one component left without redundancy: if it fails, the whole system becomes unavailable (see section 3.7).

1.3.2 Scalability

As the number of concurrent operators and incidents grows, a single API instance would become a bottleneck. The system employs a **Horizontal Pod Autoscaler** (HPA) that scales the API tier from 3 to 10 replicas based on CPU utilization, ensuring that:

- HTTP request handling capacity increases with load
- Socket.IO connections are distributed across multiple pods

1.3.3 Resource Sharing

Multiple operators must access and modify the **same shared pool of incidents** concurrently. The distributed architecture enables:

- **Concurrent read access:** all operators see the same live state of the Kanban board
- **Coordinated write access:** distributed locks ensure that only one operator can claim a ticket at a time, while optimistic locking prevents conflicting updates

1.3.4 Geographic Distribution (Potential)

While the current deployment targets a single Kubernetes cluster (e.g., Minikube or a single cloud region), the architecture is designed to support geographic distribution. The MongoDB Replica Set could be extended with nodes in different data centers, and the API tier could be replicated across regions behind a global load balancer, enabling operators in different locations to collaborate on the same incident pool.

2 Requirements Elicitation and Analysis

2.1 Functional Requirements

FR1 Board access. An operator can join the board by registering a username/password pair and, on subsequent visits, logging in with those credentials.
Acceptance criterion: submitting valid credentials on the login form transitions the client from the login screen to the dashboard and establishes a persistent, authenticated WebSocket connection.

FR2 Incident creation. An operator can create a new incident by providing a title; the system assigns it a default status of `open` and the creator's identity.
Acceptance criterion: The new incident appears as a card in the "Open" column of every connected client without any client issuing a manual refresh.

FR3 Real-time board synchronization. All connected operators must observe incident creations, claims, releases, resolutions, and escalations as they happen.
Acceptance criterion: an action performed by operator A becomes visible on operator B's board without B reloading the page or polling.

FR4 Ticket claiming (mutual exclusion). An operator can claim an `open` or `escalated` incident for exclusive handling; once claimed, no other operator can claim or resolve it.
Acceptance criterion: of two concurrent claim requests on the same incident, exactly one succeeds (`200`) and the other is rejected (`409`, reporting the current owner).

FR5 Explicit claim release. An operator holding a claim can release it without resolving the incident, returning it to the shared pool for other operators.
Acceptance criterion: after a successful release, the incident becomes claimable again immediately, without waiting for the automatic lock expiry.

FR6 Ticket resolution. The operator currently holding the claim on an incident can mark it as `closed`, recording who closed it and releasing the associated lock.
Acceptance criterion: a resolved incident moves to the "Resolved" column on every client and can no longer be claimed or resolved again.

FR7 Automatic SLA escalation. Incidents left in the `open` status beyond a configured threshold (5 minutes) are automatically transitioned to `escalated` without operator intervention.
Acceptance criterion: an incident older than the threshold changes to `escalated` within one execution cycle of the background task ($\leq$ 10 seconds after threshold crossing) and moves to the "Escalated" column on every client.

FR8 Presence awareness. The system displays the set of operators currently connected to the board.
Acceptance criterion: the online-users list reflects a new connection within one broadcast cycle, and a user disappears from the list once their connection is terminated (including ungraceful terminations).

FR9 Filtering and search. An operator can narrow the visible incidents via free-text search on the title, an "escalated only" toggle, and a "mine only" toggle (incidents they created, claimed, or resolved).
Acceptance criterion: activating any filter combination updates only the Kanban columns; the KPI summary counters continue to reflect the true, unfiltered system state.

FR10 Event notifications. Operators receive non-blocking toast notifications for relevant events they did not personally trigger, and for local operation failures.
Acceptance criterion: every failed claim, release, or resolve action produces a visible notification describing the reason for failure; no user-initiated action fails silently.

2.2 Non-Functional Requirements

NFR1 No lost updates. Concurrent modification attempts on the same incident must never silently overwrite one another.
Realized by: optimistic locking via a `version` field, matched on every `PATCH`.
Acceptance criterion: a `PATCH` carrying a stale version is rejected with `409` and leaves the document unmodified.

NFR2 Fault tolerance of the API tier. The loss of any single API pod must not interrupt service for connected or new operators.
Acceptance criterion: deleting one of the ≥ 3 running `api` pods produces no visible downtime; remaining pods continue serving requests, and Socket.IO clients transparently reconnect to a healthy pod.

NFR3 Fault tolerance of the data tier. The loss of any single MongoDB node must not lose committed data, and must restore read/write availability automatically, within a primary election.
Realized by: a 3-node Replica Set with automatic primary election, protected by a `PodDisruptionBudget` (`minAvailable: 2`) that prevents voluntary evictions from breaking quorum.
Acceptance criterion: terminating one of the 3 `mongo` pods triggers automatic recovery (re-election if the primary was lost) without manual intervention, and the API resumes normal operation once the probe reports readiness again.

NFR4 Horizontal scalability. The API tier must absorb increased load by adding replicas automatically, without manual intervention.
Realized by: a `HorizontalPodAutoscaler` (3–10 replicas, 80% CPU utilization target).
Acceptance criterion: sustained CPU load above the threshold results in additional `api` pods being scheduled within the HPA's stabilization window.

NFR5 Reconnection consistency. A client that experiences a temporary disconnection (e.g., during a rolling update of the API tier) must never regress to a state older than what it displayed before disconnecting, and must recover the *whole* observable state of each incident, including who currently holds its claim.
Realized by: a resynchronization-generation counter that discards any hydration response superseded by a later one, and a board snapshot that carries the claim state read from Redis rather than the MongoDB documents alone.
Acceptance criterion: rapid, repeated reconnects never leave the client displaying data fetched by an earlier, superseded hydration; and an operator who reloads

the page while holding a claim still sees that claim, and retains the actions it authorizes.

NFR6 UI eventual consistency. A given incident must never be rendered in two mutually-exclusive Kanban columns at once, regardless of the arrival order of independent real-time events describing the same state change.
Acceptance criterion: resolving a claimed incident never produces a transient duplicate card appearing simultaneously in "In Progress" and "Resolved", irrespective of whether the lock-release event or the database change-stream event is delivered to the client first.

2.3 Implementation Requirements

I1 The backend is implemented in Node.js, using Express for the REST API and Socket.IO for the WebSocket transport. This follows from the design need for a single event-driven runtime capable of hosting both request/response handling and long-lived WebSocket connections in the same process, which simplifies bridging MongoDB Change Stream events onto Socket.IO broadcasts.

I2 MongoDB is deployed as multi-node Replica Set. A one-member replica set would be sufficient to unlock the Change Streams API, but would not exercise the replication and automatic-failover mechanisms.

I3 Redis is present as the shared coordination point for all cross-instance mechanisms (Socket.IO Pub/Sub adapter, distributed locks, leader election). This follows directly from the design decision to keep the API tier stateless and horizontally scalable, which in turn requires an external, shared point of coordination reachable by every replica.

I4 The system is containerized with Docker and orchestrated with Kubernetes, validated locally on Minikube.

I5 The frontend is a Vue 3 single-page application using the Composition API.

2.4 Relevant and Non Relevant Distributed System Features

2.4.1 Transparency

Transparency is **partially relevant**. *Location and replication* transparency are provided: the browser never knows which of the 3 to 10 `api` pods serves it, hidden behind the Ingress, nor which `mongo` node is currently primary, hidden behind the replica-set connection string. *Failure* transparency is partial: after a pod failure the client reconnects and resynchronizes on its own instead of surfacing an error.

2.4.2 Fault Tolerance, Dependability

Fault tolerance is **relevant**. The system must survive the loss of any one `api` pod (stateless, ≥ 3 replicas) and of any one `mongo` node (3-node Replica Set) without interrupting service, and must never lose or silently corrupt data, which is why concurrent writes use optimistic locking rather than "last write wins". Recovery is automatic: if the leader fails, its 15-second lock expires and another pod resumes the escalation task within seconds. Redis is the deliberate exception: a single, non-persistent instance holding only coordination state. Its loss costs no incident or credential, but makes the whole board unavailable until it returns, and a restart drops all active claims.

2.4.3 Scalability

Scalability is **relevant** for the stateless API tier and **out of scope** for the data and coordination tiers. The `api` Deployment scales between 3 and 10 replicas through a `HorizontalPodAutoscaler` on CPU utilization.

2.4.4 Security, Trust

Security is **deliberately minimized** in favour of the project's focus on distributed coordination. Operators authenticate with a username and password.

2.4.5 Resource Sharing

Resource sharing is the **central motivation** for the entire project: multiple SOC operators must concurrently read and modify the *same* shared pool of incidents, which is precisely the scenario that requires distributed coordination rather than a simple single-user application. This single feature justifies two of the three coordination mechanisms implemented: the distributed lock, which serializes exclusive access to an individual incident (claim/release), and optimistic locking, which detects (rather than prevents) conflicting concurrent writes on the shared MongoDB collection.

2.4.6 Evolvability, Maintainability

This feature is **relevant**. The board will need updates after deployment, such as new incident fields or states, and rolling updates replace the `api` and `web` replicas gradually, without stopping the service.

2.4.7 Performance, Concurrency, Computation/Communication Efficiency, Bandwidth

This feature is **relevant**, though no quantitative target was elicited. For communication efficiency, updates are pushed through MongoDB Change Streams and Socket.IO over WebSocket rather than by polling. The main trade-off is that every `api` pod opens its own Change Stream cursor: each pod then emits only to its local clients, with no

duplicate delivery through the Redis adapter, but the primary serves one cursor per pod, a cost that grows linearly as the HPA scales toward 10 replicas.

2.4.8 Economy, Costs

This feature is **not a primary driver** of the design, since the project targets a local Minikube cluster for development and academic validation rather than a production deployment with a defined operating budget.

3 Design

3.1 Architecture

The system follows a three-tier, layered architecture (presentation, application, data), combined with an event-driven / publish-subscribe style for state propagation. It is deliberately *not* split into microservices: the four containers are tiers, not services. `web` only serves static files, `mongo` and `redis` are backing stores shared by every replica, and all domain logic lives in a single `api` service, which hosts the REST endpoints, the WebSocket server, the Change Stream consumer and the escalation task. The WebSocket channel is push-only and is fed directly, inside the same process, by the REST handlers and the Change Stream; splitting them would add inter-service coordination without a benefit at this scale.

Two motivations drive this style, both traceable to section 2:

- **Horizontal scalability**: the `api` tier is stateless *with respect to domain state*, which lets the HPA scale it between 3 and 10 replicas. Any request can be served by any replica, since identity travels in a JWT and domain state lives only in MongoDB (incidents and operator accounts) and Redis (claims, leadership). Each replica still holds disposable *connection* state — its WebSocket sessions and its Change Stream cursor — which a client simply rebuilds by reconnecting elsewhere and rehydrating.
- **Near-real-time propagation**: instead of polling, clients receive events through two channels. Changes to incidents are observed by *every* replica through its own MongoDB Change Stream and pushed to that replica's clients, so the data layer itself is the source of truth for *what changed*; changes to ephemeral state (claims, presence) start on one replica and reach the others through Redis Pub/Sub, which acts as the message broker.

3.2 Infrastructure

The infrastructure is composed of the following components, all containerized and orchestrated by Kubernetes (validated on Minikube):

- **Clients:** operator web browsers, running the Vue.js 3 SPA.

- **Static file server:** an Nginx container (`web` Deployment) serving the compiled Vue.js bundle; stateless, two replicas protected by a `PodDisruptionBudget`.
- **Application servers:** Express.js + Socket.IO processes (`api` Deployment), stateless with respect to domain state, horizontally scaled 3–10 replicas by a `HorizontalPodAutoscaler` on CPU utilization, and protected by a `PodDisruptionBudget` (`maxUnavailable: 1`, a relative bound that adapts to the current replica count).
- **Reverse proxy / load balancer:** the Kubernetes Ingress (Nginx controller, enabled as a Minikube addon), which performs path-based routing (`/` to `web`, `/api/*` and `/socket.io/*` to `api`) and cookie-based sticky sessions across the `api` replicas.
- **Database:** a 3-node MongoDB Replica Set (`StatefulSet`), the durable system of record for both incidents and operator credentials (the `incidents` and `users` collections). Each member owns a `PersistentVolumeClaim` created from the StatefulSet's `volumeClaimTemplates`, so its data survives the recreation of its pod, and a `PodDisruptionBudget` (`minAvailable: 2`) prevents voluntary disruptions from taking away the majority needed to elect a primary.
- **Message broker / coordination store:** a single Redis instance, serving three distinct roles simultaneously — the Socket.IO Pub/Sub adapter (cross-pod WebSocket broadcast), the distributed-lock store (incident claims), and the leader-election store (background task). It runs as a one-replica Deployment with no persistence and no probes of its own, which makes it the only component without redundancy: since every `api` readiness probe pings Redis, its failure takes *all* `api` replicas out of the Service at once. This single point of failure is a deliberate simplification of the project, not an oversight.

Network placement. In the validated deployment, all components run within a single Kubernetes cluster (a single-node Minikube instance), but each is an independently scheduled Pod communicating exclusively over the cluster's internal network. The architecture therefore does not preclude spreading these components across multiple nodes in a production-grade cluster; this was intentionally left as a deployment-time concern rather than an application-level one.

Component discovery. Services are located through Kubernetes, with no custom mechanism in the application code:

- `redis` is reached through its `ClusterIP` Service and the cluster DNS (`redis`, i.e. `redis.app.svc.cluster.local`). The `api` and `web` Services, instead, are used by the Ingress only as endpoint lists: it forwards external traffic directly to the pods, which is what allows its affinity cookie to pin a client to one `api` pod.

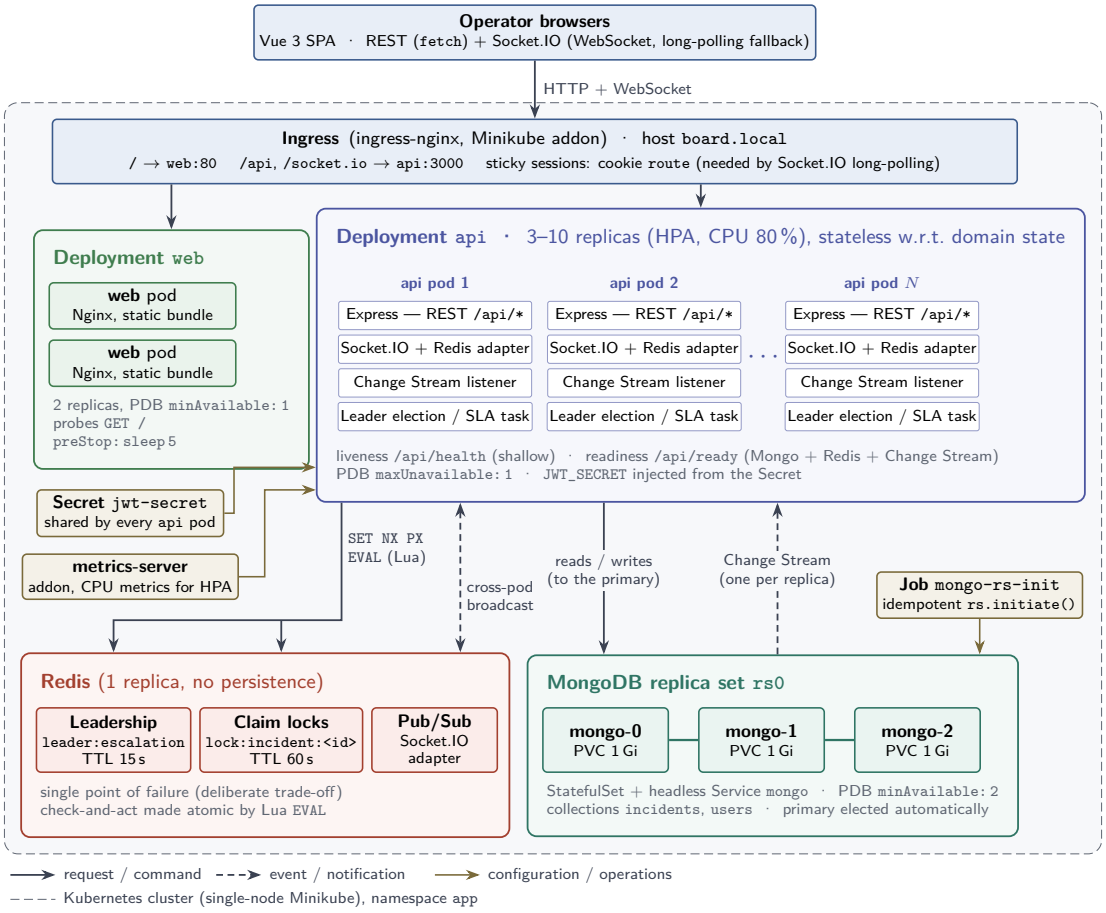


Figure 1: System architecture on the Kubernetes cluster.

- MongoDB uses a *headless* Service with the StatefulSet's stable per-pod names (`mongo-0.mongo`, ...), because replica-set members must be reachable individually rather than load-balanced.
- Externally, the Ingress exposes a single hostname (`board.local`) with path-based routing.

Beyond this, the MongoDB driver discovers the current primary by itself from the seed list in `MONGO_URL`, and the `api` replicas never address each other: they meet only through the shared stores — Redis (adapter channels, `io.fetchSockets()`, the `lock:incident:<id>` and `leader:escalation` keys) and MongoDB (the `version` field checked by optimistic locking) — so the HPA can add or remove replicas without reconfiguring the others.

3.3 Modelling

Domain entities. The core entity is the **Incident** (`title`, `status` $\in$ {`open`, `escalated`, `closed`}, `createdBy`, `closedBy`, `version`, `updatedAt`). Two further concepts are kept outside it. The **Operator** has persistent credentials (username and password), but its *presence* exists only while its WebSocket connection is open. The **Claim**, the exclusive right to work on an incident, is never written to MongoDB: it is a Redis key with a time-to-live, which is nevertheless read back into the board snapshot at every hydration. Figure 2 shows these entities together with the backend modules that operate on them.

Mapping to infrastructure. This distinction directly reflects each entity's durability requirement:

- The **Incident** must survive pod restarts, rolling updates, and member failures, and is therefore stored in MongoDB, replicated across the 3-node Replica Set.
- The **Claim** and the **leadership mandate** are, by design, short-lived and self-expiring; persisting them durably would be both unnecessary and actively harmful (a durable lock that outlives its holder's crashed connection would deadlock the resource). They are therefore stored in Redis, an in-memory store with native TTL support.
- The **Operator**'s *credentials* are persisted in MongoDB, because authentication must outlive any single connection or pod; the operator's *presence*, by contrast, is not persisted anywhere: it exists only as `socket.data.username` on a live connection, discoverable cluster-wide on demand via `io.fetchSockets()`, and simply ceases to exist the moment the connection closes.

Domain events. `IncidentCreated`, `IncidentClaimed`, `IncidentReleased`, `IncidentResolved`, `IncidentEscalated`, and `PresenceChanged` are the events that drive the system. They are realized, respectively, as: MongoDB Change Stream notifications for the events that mutate the Incident's persistent state (created, resolved, escalated), and direct Socket.IO broadcasts for the events that mutate only ephemeral Redis state (claimed, released, presence changed).

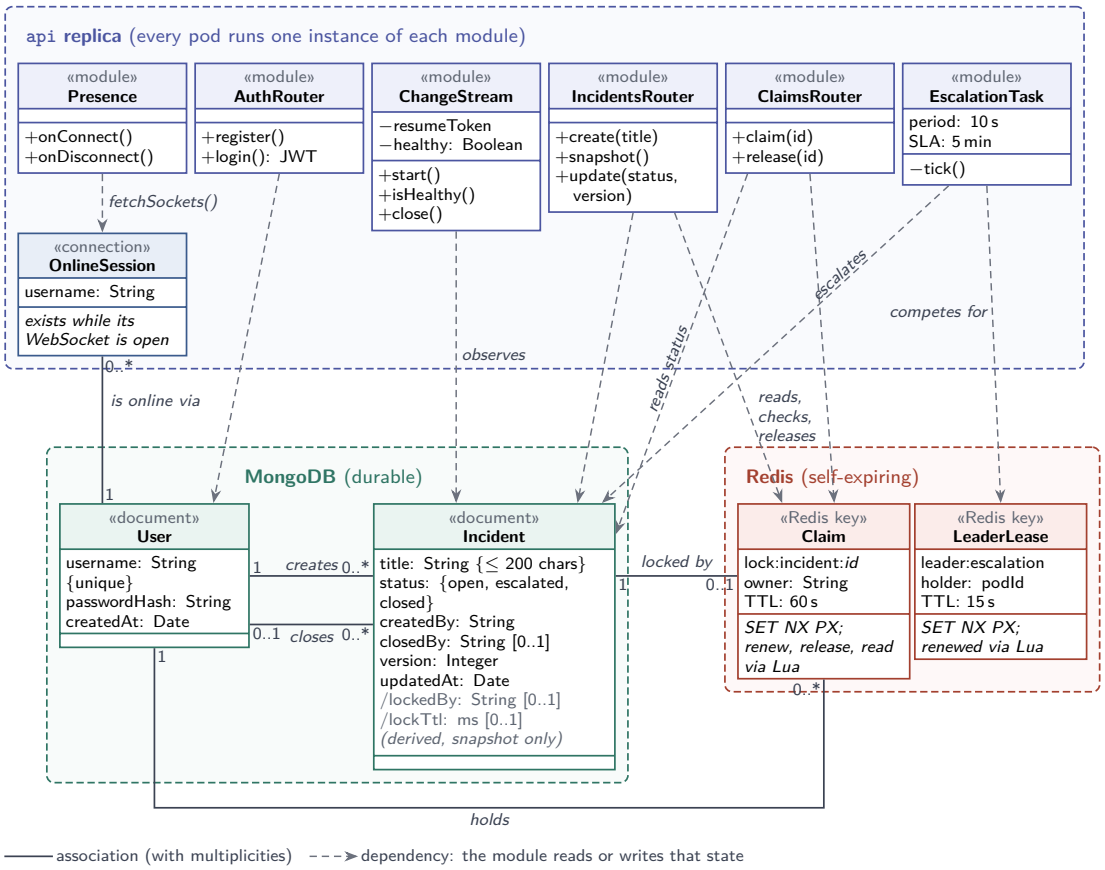


Figure 2: Class diagram.

Messages. Two distinct message categories are exchanged:

- **commands** (client → server, over REST: `POST /incidents`, `POST/DELETE .../claim`, `PATCH /incidents/:id`), which expect a definite success/failure reply and change system state.
- **events** (server → client, over Socket.IO: `incident_update`, `incident_locked/incident_unlocked`, `users_update`, `resync_required`), which are fire-and-forget notifications with no expected reply. There are no *queries* beyond `GET /api/incidents`, used to hydrate a client's local state at connection time, after every reconnect, and whenever the server itself asks for a resynchronization via `resync_required` (section 3.7) — after hydration, the client never polls again, relying entirely on the event stream to stay current.

System state. The authoritative state of the system is split across two stores by design: MongoDB holds the durable Incident collection; Redis holds transient coordination state (locks, leadership). Each connected client additionally holds a *derived*, client-local copy of a filtered view of that state (the `incidents` and `onlineUsers` reactive references in the Vue application), which is not authoritative and is kept eventually consistent with the server through the event stream and the reconnect-time resynchronization.

3.4 Interaction

Two interaction patterns coexist, cleanly separated by direction:

- **Client → server: request/response.** Every mutation (create, claim, release, resolve) is a synchronous HTTP call that yields a definite outcome (`2xx` success or `4xx`/`5xx` failure) before the client considers the action complete.
- **Server → client: publish/subscribe.** All state changes are broadcast to every interested client without any client explicitly requesting them, and without the server tracking who has or hasn't yet seen a given update. No server-initiated request ever targets one specific client.

Within the publish/subscribe direction, two further sub-patterns are used depending on the origin of the change:

- Changes to **persistent** state (create, resolve, escalate) are captured by MongoDB Change Streams and broadcast via `io.local.emit()` because every `api` pod independently watches the same Change Stream and would otherwise re-broadcast an event that every other pod has *already* independently received and forwarded to its own clients, causing duplicate delivery.
- Changes to **ephemeral** state not reflected in MongoDB (claim, release, presence) are broadcast via `io.emit()`, which *does* need to cross pod boundaries through the Redis adapter, since only one pod knows the change happened: the one that served the claim or release request, or the one where an operator's WebSocket

connected or disconnected. Resolving an incident uses both channels at once: the status change reaches clients through the Change Stream, while the release of its lock is announced with `io.emit()`.

Sequence diagrams. The four most significant interactions are shown below, each with two `api` pods so that it is visible which messages cross pod boundaries. Figure 3 covers login, the WebSocket handshake, presence and the hydration of the board. Figure 4 shows a claim with its four possible outcomes. Figure 5 shows a resolution, where the two channels run in parallel: the release of the lock travels through Redis Pub/Sub, the status change through the Change Stream of every pod. Figure 6 shows the SLA escalation run by the leader, which writes to MongoDB and leaves the notification to the Change Streams.

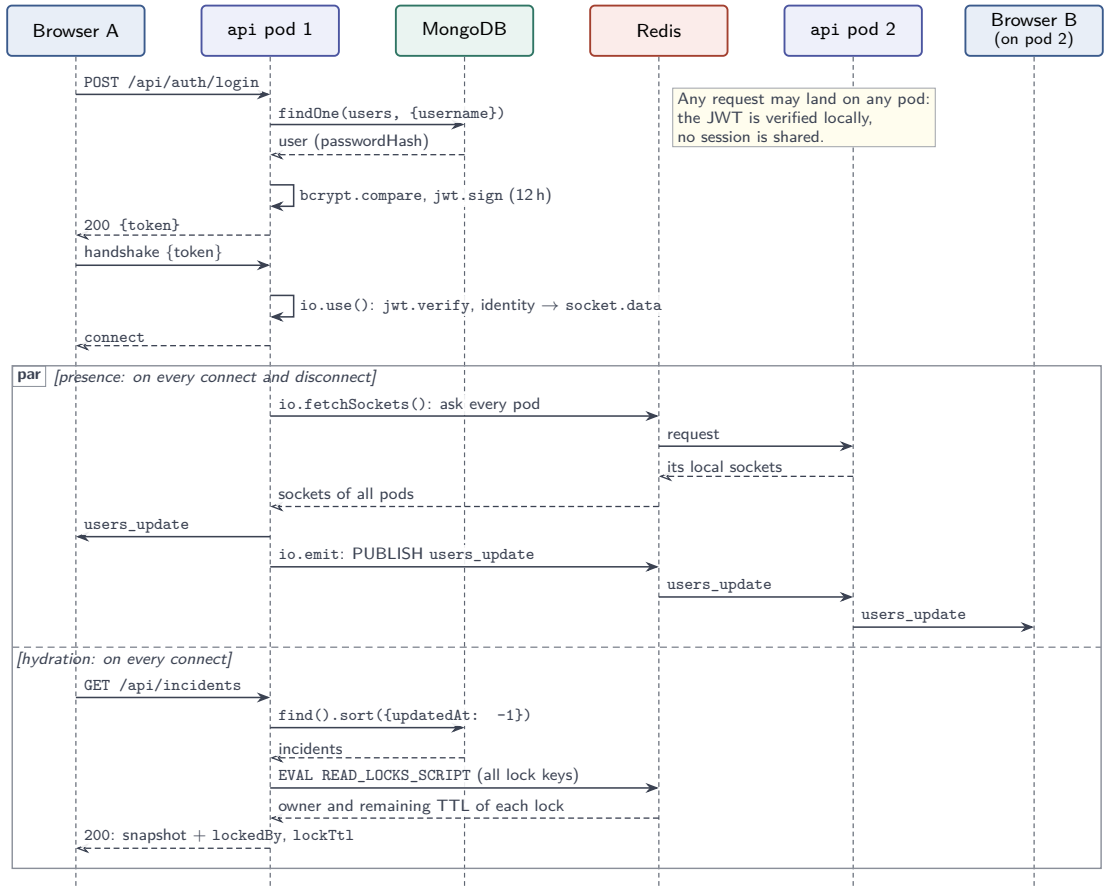


Figure 3: Login, WebSocket handshake, presence and hydration. Presence and hydration both start at every connection and run concurrently.

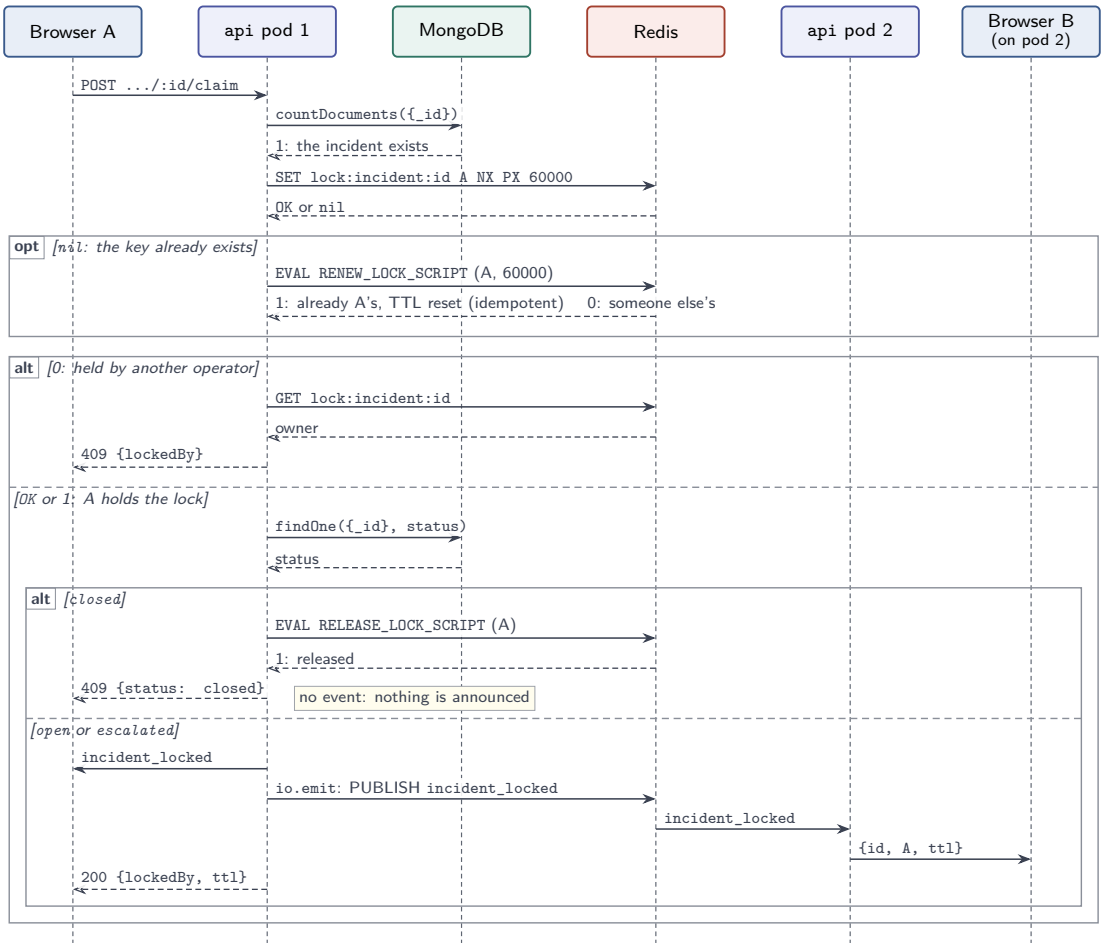


Figure 4: Claiming an incident. The claim is idempotent for the operator who already holds the lock; the `409` is reserved for a conflict with another operator or an already resolved incident.

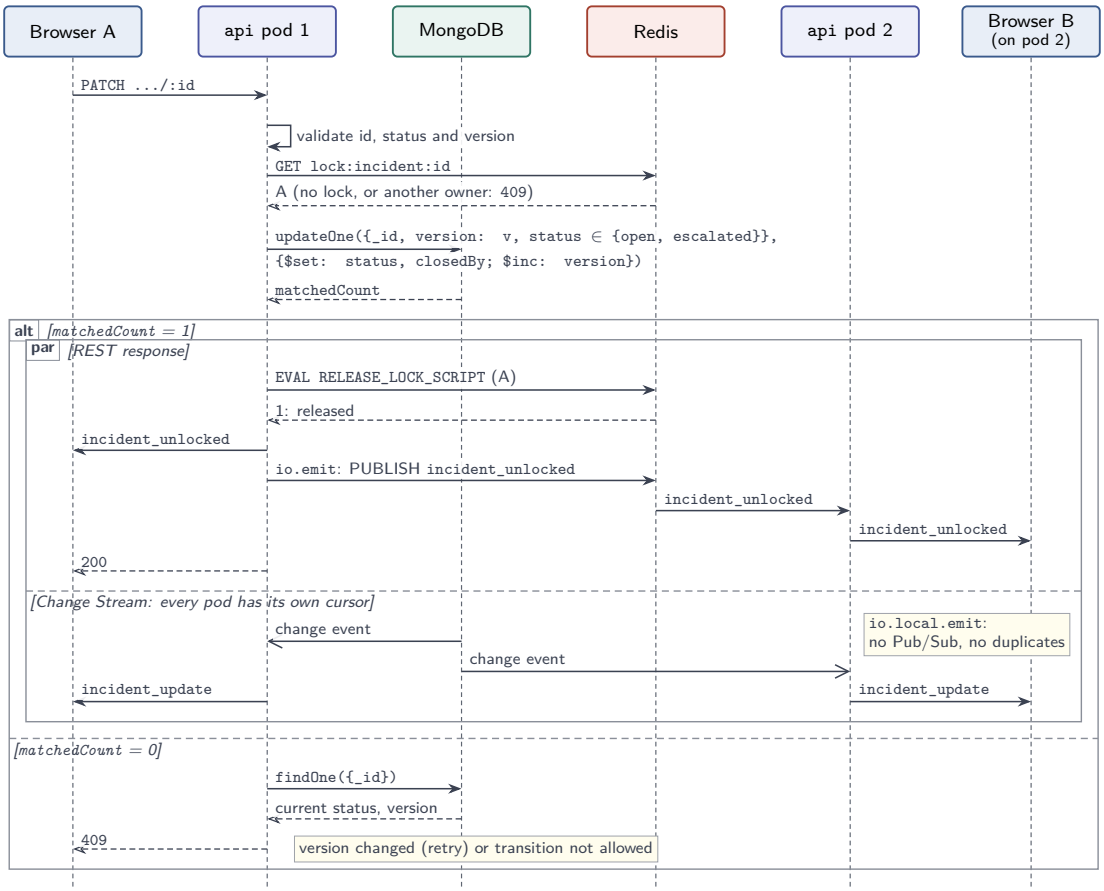


Figure 5: Resolving an incident. The `PATCH` requires owning the lock, and the update is conditioned on both the version and the allowed source status.

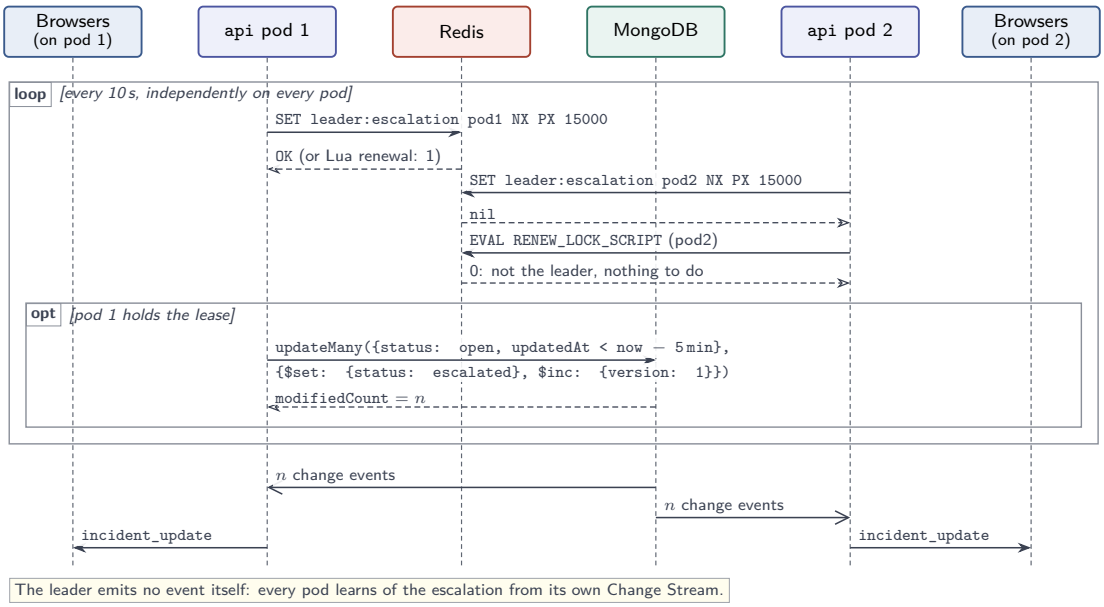


Figure 6: SLA escalation. Every pod competes for the lease at every tick; only the leader writes, and every pod notifies its own clients from its Change Stream.

3.5 Behaviour

- **`api` pods** keep no domain data in memory and react to four stimuli: REST requests, handled synchronously; WebSocket connections and disconnections, which recompute the presence list with `io.fetchSockets()`; events from their own Change Stream, forwarded only to their local sockets; and a self-generated 10-second timer that drives the leader-election attempt and, on the leader, the escalation scan.
- **MongoDB** accepts writes only on the primary, which replicates them to the secondaries through the oplog; if the primary becomes unreachable, the majority elects a new one with no application intervention.
- **Redis** is purely reactive: it executes the commands of the `api` pods (`SET NX PX`, `GET`, `EVAL` of the Lua scripts), relays the adapter's Pub/Sub messages, and expires keys on its own when their TTL elapses.
- **The Vue.js client** never updates its state optimistically: the response to a command only reports its outcome, and the board changes only through server events and snapshots, so it never shows the effect of a command before the server has confirmed it.

State diagrams. Three components have an explicit lifecycle worth modelling. An **incident** (fig. 7) combines its durable status in MongoDB with the ephemeral claim in Redis: the two dimensions are independent, which is why an incident can be escalated

while someone is working on it. Each `api` pod has an internal state machine for its role in the **leader election** (fig. 8). Finally, each **client session** (fig. 9) moves between connecting, hydrating from the REST snapshot and following live events.

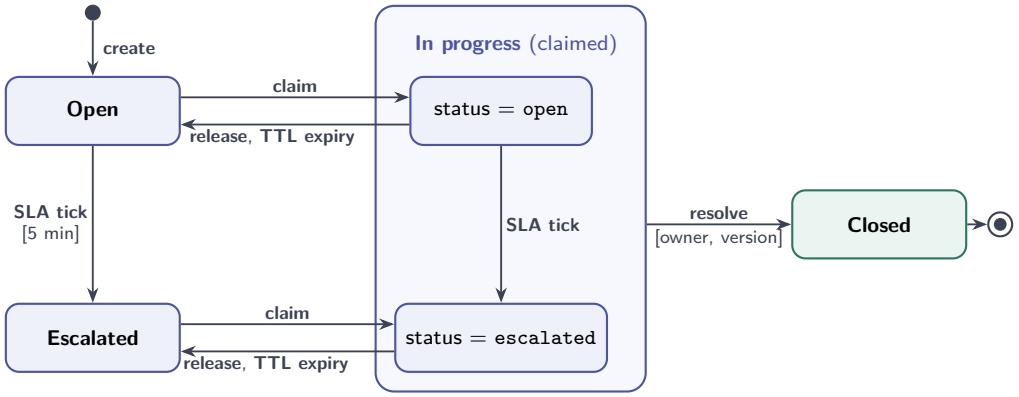


Figure 7: Incident lifecycle. The *In progress* composite state corresponds to the claimed incidents; its two substates keep the status that the incident returns to when the claim is released or expires.

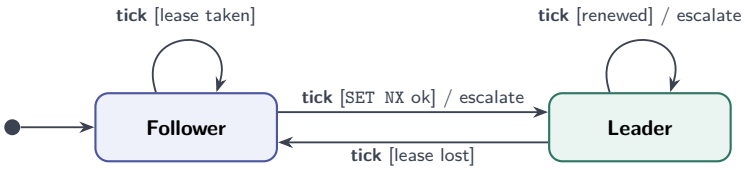


Figure 8: Leader election, as seen by a single `api` pod at every 10-second tick.

Disconnection and cleanup. A client leaves voluntarily (logout closes its socket), through a network failure, or by crashing, which the server notices only when the Socket.IO heartbeat times out. In every case the pod handles the `disconnect` by recomputing presence with `io.fetchSockets()` and broadcasting `users_update`. Nothing else has to be cleaned up, because no per-client state is stored anywhere: presence is derived from the live sockets, and claims are deliberately *not* released on disconnect. A claim belongs to the operator, not to the connection, so a reload or a brief network drop does not cost it, while its 60-second TTL bounds how long an abandoned one can block the incident; logging out likewise leaves held claims to expire.

3.6 Data and Consistency Issues

What is stored, and where. Incidents and operator accounts are the only data that must persist beyond a single request or connection; they are stored in **MongoDB** as

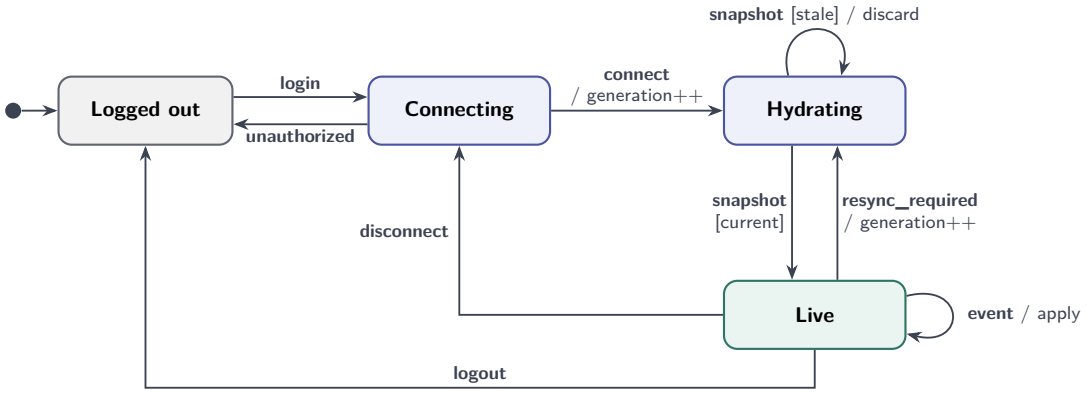


Figure 9: Client session in the browser. The *generation* counter discards snapshots that arrive after a newer resynchronization has started.

documents, in the `incidents` and `users` collections. The document model was chosen because incidents are naturally semi-structured records whose schema evolved incrementally over the project's development (e.g., the `escalated` status and the `closedBy` field were both introduced after the initial design) without requiring a formal migration step. Coordination data (locks, leadership) is stored in **Redis** as plain key-value pairs, chosen because the access pattern is a pure read/write/expire on independent keys with no relational structure, and because lock contention needs sub-millisecond latency that an in-memory store provides naturally.

Query and write patterns. `api` pods are the only components that ever query or write MongoDB and Redis; the client never talks to either data store directly, always mediating through the REST/Socket.IO layer. Concurrent *reads* are unrestricted and require no coordination (both MongoDB and Redis handle concurrent reads natively). Concurrent *writes* are expected and must be explicitly coordinated: this is precisely the motivation behind the two concurrency-control mechanisms of section 2 — optimistic locking (NFR1) detects conflicting concurrent writes to the *same* Incident document, and the distributed lock (FR4) prevents two operators from concurrently claiming the same Incident in the first place.

Shared data. The `incidents` and `users` collections are, by design, shared read/write state across every `api` replica (which is why the uniqueness of usernames is enforced by a unique index rather than by any single replica, section 3.9) — this sharing is the entire point of keeping the application tier stateless and horizontally scalable. The same is true of the Redis keyspace: coordination only works if every replica observes the same lock and leadership state, which is why a single shared Redis instance (rather than one per pod) is a hard requirement, not an implementation convenience.

Consistency model. The system provides **strong consistency** for writes to the primary (a successful write is immediately visible to any subsequent read against the primary) and **eventual consistency** for propagation to connected clients. A change to an incident is always durably committed to MongoDB *before* any client is notified of it, since the Change Stream reports only majority-committed writes, but different clients may observe the notification at slightly different times depending on Change-Stream and network latency. Claims and presence take the other channel, the Redis Pub/Sub of the Socket.IO adapter: a claim is announced as soon as Redis has accepted it, and since Redis is not persistent it is volatile coordination state rather than a durable record; the natural expiry of a claim produces no event at all and is predicted by each client from the remaining TTL. This asymmetry is a deliberate, accepted trade-off: the domain (a SOC dashboard) tolerates sub-second staleness in what operators *see*, but never tolerates staleness or loss in what is actually *recorded* as the incident's state.

3.7 Fault-Tolerance

Replication. MongoDB runs as a 3-node Replica Set. The loss of a secondary goes unnoticed; the loss of the primary costs no acknowledged write, since MongoDB 6 acknowledges a write only once a majority of the nodes holds it (its default write concern), but it suspends reads and writes until the two remaining nodes elect a new primary — about 15 s in the acceptance test (section 5.1), during which the `api` readiness probes may fail. Redis is deliberately *not* replicated (section 2.4, *Fault Tolerance, Dependability*): it holds coordination state, not the system of record, so its failure costs availability while it is down and the active claims, which no one reconstructs, but never durable data; the leadership is simply re-elected at the next tick. The 3–10 `api` pods and the 2 `web` pods are interchangeable workers, stateless with respect to domain state, rather than replicas of any data.

Heartbeats, timeouts, and retries.

- **Probes.** The liveness probe (`/api/health`) is shallow and the readiness probe (`/api/ready`) is deep: it checks MongoDB, Redis and the pod's own Change Stream. A failing dependency therefore takes a pod out of the Service without restarting it.
- **Leases.** Claims and leadership are Redis keys with a TTL: a crashed holder needs no failure detector, because its lock simply expires.
- **Connection heartbeat.** Socket.IO's ping/pong detects dead connections, which keeps the presence list computed with `io.fetchSockets()` correct.
- **Client retries.** The Socket.IO client reconnects automatically, and the resynchronization-generation counter (NFR5) discards out-of-date snapshots.
- **Change Stream recovery.** A broken stream is rebuilt with exponential backoff (1 s doubling up to 30 s) and `resumeAfter`, so the changes made during the outage

are replayed. If the resume token is no longer in the oplog, the stream restarts from the present and the pod sends `resync_required` to its clients. If the stream is still down after 20 s, the pod disconnects its clients so that they reconnect to a healthy pod.

Error handling. Four classes of failure are handled, each by the cheapest mechanism that recovers from it:

- **Backend crash.** An `api` pod that dies is restarted by Kubernetes, and one that hangs is restarted when its shallow liveness probe fails. Nothing is lost with it: claims and leadership live in Redis, and the leadership passes to another pod once the 15-second lease expires. Its WebSocket clients reconnect through the Ingress to another replica and rehydrate, while a REST request in flight fails with an error from the Ingress (typically a `502`), which the client reports like any other server error; a mutation may already have been applied before the crash, in which case its effect still reaches every client through the Change Stream. A failure to reach MongoDB or Redis *at startup*, instead, is fatal by design (`process.exit(1)`): the pod is restarted rather than serving traffic half-initialized.
- **Redis failure.** The Redis clients reconnect on their own, and the commands issued in the meantime wait in their offline queue. The readiness probe's `PING` therefore times out, every `api` pod leaves the Service without being restarted, and the board stays unavailable until Redis returns; after a restart the active claims are lost, and the leadership is re-elected at the next tick (section 2.4).
- **MongoDB failure.** The loss of a secondary is invisible. The loss of the primary suspends reads and writes until the election: the driver waits for the new primary, retries a failed read or write once, and resumes the Change Streams from their tokens on its own; the errors it cannot resume are left to the pod's Change Stream recovery described above. If the whole Replica Set is down, readiness fails on every pod, as with Redis.
- **REST API errors.** Every REST handler catches its errors and answers with a specific status — `400`/`404` for invalid input, `401` for a missing or invalid token or wrong credentials, `409` for a lock, version or transition conflict or an already registered username, `500` otherwise — instead of crashing the process. On the frontend, every action checks the response and shows a notification on failure, distinguishing the server's message from a network error and tolerating bodies that are not JSON (e.g., a `502` from the Ingress); a token rejected at the WebSocket handshake sends the operator back to the login form.

3.8 Availability

Caching. No server-side cache is used for Incident data: every `GET /api/incidents` reads MongoDB directly. A cache would buy little: clients call it only at login, on

reconnection and on `resync_required`, and live on events in between, so the read volume does not grow with the use of the board. The snapshot also carries each claim's remaining TTL, read from Redis at that instant, which a cache would serve stale. The role of a cache is played instead by each client, which keeps the whole board in memory and updates it from the event stream. Redis, despite superficially resembling a cache, is never used to cache MongoDB reads; its only roles are coordination (locks, leadership) and message brokering (the Socket.IO adapter).

Load balancing and sticky sessions. The Ingress load-balances HTTP and WebSocket traffic across the ≥ 3 `api` replicas with cookie-based **sticky sessions** (cookie `route`). Socket.IO requires them: every connection opens on HTTP long-polling and only then upgrades to WebSocket, and the handshake, the polling requests and the upgrade must all reach the *same* process, which holds the session. Affinity concerns this connection state only and does not contradict the stateless API tier: REST calls carry the cookie too, since the Ingress rule is shared, but any replica could serve them, because identity travels in the JWT. The cookie is a preference rather than a binding: a pod that fails or turns unready leaves the endpoint list, and the Ingress moves its clients to another replica. This is also why a pod whose Change Stream is down disconnects its clients only after 20 s (section 3.7): done earlier, the cookie would send them straight back to the same pod. The `HorizontalPodAutoscaler` keeps the pool of backends elastic under load (section 2.2).

Behaviour under network partitioning. In every scenario below the system favours **consistency** over the availability of the partitioned side:

- If an `api` pod loses connectivity to Redis or MongoDB, or its Change Stream cannot be re-established, `/api/ready` starts failing and the pod leaves the Service's endpoints without being killed, rejoining once connectivity returns; new requests reach only pods that can serve them correctly. Its open WebSockets are moved only when the Change Stream is what failed (after 20 s): a pod cut off from Redis alone keeps its clients, which miss the lock and presence events published meanwhile, a divergence bounded by the claims' 60-second TTL.
- If the leader is cut off from Redis, it can no longer renew its lease and stops escalating, and another pod takes over within about 25 s (the 15 s lease, plus up to one 10 s tick). If it is cut off from MongoDB alone, it keeps the lease while its escalation scans fail, so escalation is suspended cluster-wide until the partition heals: no data is lost, since overdue tickets are escalated at the next successful scan.
- A MongoDB secondary partitioned from the primary falls behind and resynchronizes once the partition heals; clients do not notice, since every read goes to the primary.

- A primary partitioned from the majority steps down, so the minority side serves neither reads nor writes rather than risk divergent incident data (**CP**). The writes it accepted but could not replicate were never acknowledged, since acknowledgement requires a majority, and are rolled back when it rejoins.

3.9 Security

Authentication. Operators register with a username and a password of at least 8 characters; the password is stored only as a bcrypt hash, and the uniqueness of the username is enforced by a unique index rather than by an application-level check, which would race across replicas. Login answers with the same error for an unknown user and a wrong password, so its message does not tell the two cases apart (usernames can still be enumerated through registration, which reports a name already taken, and through the response time, since no hash is compared for an unknown user), and issues a JWT (subject: the username, validity: 12 hours) signed with a secret shared by every `api` replica through a Kubernetes `Secret`. The token is required on every REST call (`Authorization: Bearer`) and on the Socket.IO handshake, and any pod can verify it locally: no session state is kept on the server. The browser keeps it in `sessionStorage`, so it survives a page reload but not the closing of the tab. The known limits are deliberate simplifications: authentication is single-factor, there is no password reset, account lockout or rate limiting on login, and the WebSocket token is checked only at the handshake, so an open connection outlives the expiry of its token.

Authorization. There is a single role: every authenticated operator can create incidents and claim any unclaimed one that is not yet resolved. Releasing and resolving, instead, require *owning* the incident's Redis lock: the server answers `409` both when the lock belongs to someone else and when there is no lock at all, so the mutual exclusion cannot be bypassed by simply not claiming. The lock therefore acts as a capability for a single incident, and it is anchored to a verified identity: its value, like `createdBy` and `closedBy`, is always the username taken from the caller's JWT and never from the request body.

4 Implementation

4.1 Technology Stack

- **Frontend:** a single-page application in Vue 3 (Composition API, `<script setup>`), built with Vite 8. `socket.io-client` 4 keeps the real-time channel, Chart.js 4 (through `vue-chartjs` 5) draws the dashboard chart, and `vue-toastification` 2 shows the notifications.

- **Backend:** Node.js 20 with Express 5 for the REST API and Socket.IO 4 for the WebSocket server, extended across pods by `@socket.io/redis-adapter` 8. Authentication uses `jsonwebtoken` and `bcryptjs`; the stores are reached through the official `mongodb` 7 and `redis` 6 (node-redis) drivers.

- **Database:** MongoDB 6 as a 3-member replica set, the durable store of incidents and users and the source of the Change Streams; Redis 7 as a single in-memory instance for the claim locks, the leader lease and the Socket.IO Pub/Sub.
- **Distributed infrastructure:** Kubernetes, validated on Minikube, with an ingress-nginx Ingress and a HorizontalPodAutoscaler fed by `metrics-server` (sections 4.5.4 and 6.2).
- **Containerization:** Docker. The `api` image is based on `node:20-alpine` with production dependencies only; the `web` image is a multi-stage build that compiles the bundle on `node:20-alpine` and serves it with `nginx:alpine`. MongoDB and Redis use the official `mongo:6` and `redis:7` images.
- **API:** REST over HTTP with JSON bodies under `/api`, authenticated with a `Bearer` JWT, and server-to-client events over Socket.IO (section 4.2).

4.2 Network Protocols

Two application-level protocols coexist, each chosen for a specific interaction pattern:

HTTP/1.1 over TCP is used for every request from the client to the server: the **commands** (mutations) as well as the snapshot read used for hydration. The REST API exposed by Express.js defines nine endpoints:

- `POST /api/auth/register`, `POST /api/auth/login` — create an operator account, and exchange credentials for a JWT; the only client endpoints that do not require one;
- `POST /api/incidents` — create a new incident;
- `GET /api/incidents` — retrieve the full incident list, each document enriched with the current holder of its claim and that claim's remaining time-to-live (used at connect time, on every reconnect, and on server-requested resynchronization);
- `POST /api/incidents/:id/claim` — acquire the distributed lock on an incident;
- `DELETE /api/incidents/:id/claim` — explicitly release a held lock;
- `PATCH /api/incidents/:id` — resolve an incident (with optimistic-locking version check and server-side lock verification);
- `GET /api/health`, `GET /api/ready` — the shallow liveness and deep readiness probes, called by the kubelet rather than by the client.

HTTP was chosen because every mutation requires an unambiguous, synchronous success-or-failure response (e.g., "did I get the lock or not?"), which request/response semantics provide naturally.

WebSocket (via Socket.IO) is used for all **event** (notification) interactions from the server to connected clients. Every Socket.IO connection opens on HTTP long-polling and then upgrades to WebSocket, staying on long-polling only if the upgrade fails; this is why the Ingress needs sticky sessions (section 3.8). The server emits five event types, listed with their payload:

- `incident_update` — the raw MongoDB Change Stream event (insert or update, with the full document) describing a persistent state change;
- `incident_locked` {`id, lockedBy, ttl`} / `incident_unlocked` {`id`} — ephemeral lock state changes. The `ttl` lets each client expire the lock locally, since no event will announce its natural expiry; it is a *relative* duration in milliseconds, so the expiry does not depend on the browser's clock agreeing with the cluster's;
- `users_update` — the array of usernames currently connected;
- `resync_required` — no payload; asks the pod's own clients to re-hydrate from `GET /api/incidents`, emitted when the pod knows it has lost a segment of the change feed and cannot replay it (section 3.7).

WebSocket was chosen over polling because the design requires near-real-time propagation of every state change to every connected client.

4.3 In-Transit Data Representation

JSON is the only application-level serialization format. REST request and response bodies are JSON (e.g., {`"status": "closed", "version": 3`}), parsed by `express.json()` and returned via `res.json()`; Socket.IO events carry the JSON payloads listed in section 4.2.

4.4 Database Querying

MongoDB is queried through the fluent API of the official Node.js driver.

Writes.

- `insertOne` creates incidents (`status: 'open'`, `version: 1`) and users, whose usernames are kept unique by an index.
- `updateOne({_id, version, status: {$in: allowed sources}}, {$set: ..., $inc: {version: 1}})` implements optimistic locking: it applies only if the document is unchanged since the client read it and the transition is allowed.
- `updateMany({status: 'open', updatedAt: {$lt: threshold}}, ...)` escalates all overdue incidents in one round-trip.

Reads. The snapshot is `find({}).sort({updatedAt: -1})`, enriched with each claim's holder and remaining TTL from Redis; documents are read before locks, so a lock acquired in between is reported rather than lost. The other reads are point lookups: the user at login, the incident before a claim and its status once the lock is acquired, and its status and version after a failed update, to tell a version conflict from a forbidden transition.

Change Streams. `watch([], {fullDocument: 'updateLookup'})` streams every insert and update with the full document. Each event's `_id` is kept as the resume token for `resumeAfter`; the driver retries resumable errors such as a failover, and the application handles the rest (section 3.7).

Redis. Besides `SET NX PX` and `GET`, three Lua scripts run atomically:

- `RENEW_LOCK_SCRIPT` extends a key's TTL only if the caller still owns it (leadership and claims);
- `RELEASE_LOCK_SCRIPT` deletes a claim only if the caller still owns it, never one that expired and was taken over;
- `READ_LOCKS_SCRIPT` returns owner and remaining TTL of every lock in one call, consistent with each other and in one round-trip regardless of board size.

4.5 Technological Details

4.5.1 Backend

- **Node.js:** a single process hosts the REST handlers, the WebSocket sessions, the Change Stream consumer and the escalation timer, which must share it because the WebSocket channel is fed by the other two (section 3.1). The workload is almost entirely I/O, so one event loop serves all of them without threads.
- **Express:** an `express.Router()` mounts every endpoint under `/api`; `express.json()` parses the bodies, and `cors()` restricts cross-origin requests to `CORS_ORIGIN`.
- **Socket.IO and the Redis adapter:** `io.emit()` is relayed to every pod through Redis Pub/Sub, `io.local.emit()` reaches only the pod's own sockets, and `io.fetchSockets()` queries all pods. The adapter needs two Redis connections, because a client in subscriber mode cannot issue other commands: `subClient`, a duplicate of `pubClient`, receives the messages, while `pubClient` publishes them and also serves locks and leadership.
- **MongoDB driver:** `MONGO_URL` lists all three members with `replicaSet=rs0`, so the driver discovers the primary by itself, follows it across elections and retries a failed read or write once.

Manifest	Resources	Key parameters
`mongo.yaml`	headless Service, ConfigMap, StatefulSet, Job, PDB	3 members `--replSet rs0`, 1 GiB PVC each; probes `ping` via `mongosh`; idempotent `mongo-rs-init` Job (`backoffLimit: 5`); PDB `minAvailable: 2`
`redis.yaml`	Deployment, `ClusterIP` Service	1 replica, port 6379, no persistence, no probes
`jwt-secret.yaml`	Secret	`JWT_SECRET`, a placeholder for the local demo
`api.yaml`	Deployment, Service, PDB	3 replicas; `MONGO_URL`, `REDIS_URL`, `CORS_ORIGIN`, and `JWT_SECRET` from the Secret; CPU request 100m, limit 500m; liveness `/api/health` every 10 s, readiness `/api/ready` every 5 s with `failureThreshold: 3`; no `preStop`; PDB `maxUnavailable: 1`
`hpa.yaml`	HorizontalPodAutoscaler	`api` between 3 and 10 replicas, 80% average CPU
`web.yaml`	Deployment, Service, PDB	2 replicas; rolling update with `maxUnavailable: 0`; `preStop sleep 5`; probes on `/`; PDB `minAvailable: 1`
`ingress.yaml`	Ingress	host `board.local`; `/` → `web:80`, `/api` and `/socket.io` → `api:3000`; cookie affinity (`route`, 48 h)

Table 1: Kubernetes manifests and their key parameters.

4.5.2 Frontend

- **Vue.js:** the interface is split into nine components below `App.vue` (header, login card, filters, KPI row, chart, Kanban board, columns, cards, creation form), while state and logic live in composables: `useIncidentsStore` holds the board and applies events and snapshots, `useRealtime` manages the socket and the resynchronizations, `useLockExpiry` expires locks locally, `useIncidentActions` sends the REST commands, `useIncidentViews` derives columns, filters and KPIs as `computed` values, and `useAuth` keeps the token.

- **socket.io-client:** the connection is opened with `io('/', {path: '/socket.io', auth: {token}})`, so it reaches the `api` pods through the same Ingress as the page, with the JWT used for REST calls. After a disconnection imposed by the server the client does not retry on its own, so `useRealtime` reconnects explicitly; otherwise an evicted client (section 3.7) would stay disconnected.

- **Chart.js** draws the status overview from the same computed KPIs, and **vue-toastification** notifies both the changes on the board (new, escalated and resolved incidents) and the outcome of the operator's own commands, distinguishing server errors from network errors.

4.5.3 Containerization

The `api` Dockerfile copies `package*.json` and runs `npm ci --only=production` before copying the code, so the dependency layer is rebuilt only when the dependencies change; in the `web` image the Node.js build stage is discarded, leaving only the static bundle on Nginx. Both images are built inside Minikube's Docker daemon and referenced with `imagePullPolicy: Never`, so no registry is involved.

4.5.4 Orchestration (Kubernetes)

Table 1 summarizes the manifests in `k8s/`, all deployed in the `app` namespace; the reasons behind probes, PodDisruptionBudgets and sticky sessions are given in sections 3.2, 3.7 and 3.8, and how they are applied in section 6.4.

Two values are tied to each other rather than to a design choice. The readiness window of `api` (5 s × 3) is shorter than the 20 s after which a pod with a broken Change Stream evicts its clients, so the pod has left the endpoints before they reconnect. The `preStop` of `web` keeps a terminating pod serving for 5 s while its removal from the endpoints propagates: without it, `maxUnavailable: 0` would preserve the capacity during a rollout but not prevent the requests refused in that window.

5 Validation

5.1 Acceptance Tests

All validation was performed by **manual testing** on the live Minikube cluster, with several browser sessions and `curl`; automating it would mean coupling browser automation with `kubectl`-driven fault injection, beyond the time budget of a single-developer project. Tests are grouped by mechanism, each listing the requirements it covers. Results were observed on the cluster unless marked *(expected)*: those steps cover behaviour introduced after the last validation round. The identity behind every write is always taken from the caller's JWT, never from the body.

5.1.1 Claim and Release (FR4, FR5)

- **Procedure:** send `POST /api/incidents/<id>/claim` concurrently with both tokens; release as the winner (`DELETE` on the same path) and claim again as the loser.
- **Expected:** exactly one claim answers `200` (with `ttl: 60000`), the other `409` naming the owner; after the release the card returns to its column everywhere and the second claim succeeds.
- **Result:** passed, since `SET NX PX` is atomic and the release emits `incident_unlocked` through the Redis adapter.

- *(Expected)* A repeated claim by the holder answers `200` and resets the TTL to about 60000 ms; the other user still gets `409`.

5.1.2 Real-Time Propagation, Mutual Exclusion and Resolution (FR2, FR3, FR4, FR6, NFR6)

- **Procedure:** while watching bob's board, create and claim an incident as alice; as bob, bypassing the UI, send `PATCH /api/incidents/<id>` with `{"status":"closed","version":1}`; then resolve it as alice from the UI.

- **Expected:** the creation answers `201`, and each step appears on bob's board within about one second, without refresh. Bob's request is rejected with `409` (`lockedBy: "alice"`) despite his valid credentials; alice's answers `200`, and the card moves to "Resolved" everywhere with alice as resolver and no further action offered, never appearing in two columns at once.

- **Result:** passed. Creation and resolution travel through the Change Stream, the claim through the Redis adapter. The server checks the lock owner against the JWT identity and releases the lock within the same request; columns are derived from both `status` and `lockedBy`, so any intermediate state falls into exactly one.

- *(Expected)* A PATCH on an unclaimed incident is also rejected with `409`, so skipping the claim does not bypass the mutex; so is a claim on the resolved incident: the server checks the status after acquiring the lock and releases it without announcing it.

5.1.3 No Lost Updates (NFR1)

- **Procedure** *(expected)***:** claim an incident as alice when it is about 4.5 minutes old, let the SLA task escalate it (`version` $1 \rightarrow 2$), then resolve it with `curl` and `version:1`, since the UI already holds `version:2` from the Change Stream.

- **Expected:** `409` with the document unchanged; retrying with `version:2` succeeds. Both requests must precede the claim's expiry, about 5.5 minutes after creation, or they are rejected for lack of a lock. This is the real race between an operator and the leader's background write.

- **Note:** the earlier procedure, two consecutive resolutions with the same version, no longer reaches the version check, because the second request is now rejected by the lock check first.

5.1.4 SLA Escalation (FR7)

- **Procedure:** with ≥ 3 `api` pods, leave an incident open for over 5 minutes, watching the boards and `kubectl logs -l app=api -n app`.

- **Expected:** within 10 s of the threshold it moves to "Escalated" on every client with a toast, its `version` grows by exactly 1, and a single pod logs it.
- **Result:** passed. Only the holder of `leader:escalation` runs the task; the Change Stream propagates the result.

5.1.5 Presence (FR8)

- **Procedure:** with alice and bob connected, close bob's tab; reconnect him, then force-delete his pod (`--force --grace-period=0`) so that no `disconnect` is emitted.
- **Expected:** "Users Online" goes from 2 to 1 and, after the forced termination, converges to the correct value with no manual cleanup.
- **Result:** passed. Presence is recomputed with `io.fetchSockets()` at every connection and disconnection, and a dead pod no longer answers it: here bob's reconnection to another pod triggered the recomputation that dropped the stale sockets. Without any later connection event, clients would keep the stale list until the next one. The earlier hand-maintained Redis hash failed this test with entries that never expired.

5.1.6 API Pod Loss and Reconnection (NFR2, NFR5)

- **Procedure:** delete one `api` pod and immediately create and claim an incident; then, with a client connected, run `kubectl rollout restart deployment/api -n app`.
- **Expected:** no visible downtime; clients reconnect on their own to a surviving pod and never show a snapshot fetched by a superseded connection.
- **Result:** passed. The client re-fetched the board on each reconnection and, during the rollout, went through 1–2 brief disconnections; the resynchronization-generation counter discards stale responses.
- *(Expected)* After a reload, alice's claimed card is still "In Progress" with her buttons, since the snapshot carries `lockedBy` and `lockTtl`; left idle, it returns to its column on every client when the TTL elapses.
- *(Expected)* A pod whose Change Stream has died answers `503` on `/api/ready` and leaves the Service without restarting; its clients are moved to a ready pod and it rejoins once the cursor is rebuilt. A non-resumable failure is hard to inject from outside, since the driver silently retries resumable ones such as a `mongo` pod deletion.

5.1.7 Data-Tier Failover (NFR3)

- **Procedure:** delete the pod of the MongoDB primary (`rs.hello().primary`) and follow `rs.status()`.
- **Expected:** a new primary is elected within 10–30 s; the `api` readiness may fail meanwhile; the old pod rejoins as a secondary.
- **Result:** passed, with the election in about 15 s. A direct deletion is not an eviction, so the PodDisruptionBudget plays no part here.

5.1.8 Horizontal Scalability (NFR4)

- **Procedure:** load the `api` pods with a tight `curl` loop and watch `kubectl get hpa api-hpa -n app -w`.
- **Expected:** above 80% average CPU the HPA adds replicas, up to 10.
- **Result:** passed, scaling from 3 to 5 replicas.

5.1.9 Client-Side Checks (FR1, FR9, FR10)

Checked directly in the browser, as they involve no distributed mechanism:

- **FR1** — registration and login open the dashboard with an authenticated WebSocket; invalid credentials and duplicate usernames are rejected by the server. *Passed.*
- **FR9** — search and toggles filter the Kanban columns, not the KPI counters. *Passed.*
- **FR10** — creations and resolutions by others, escalations and every failed action produce a toast, failures with the server's reason. *Passed* for claim, release and resolve; incident creation now reports failures too *(expected).*

6 Deployment

6.1 Prerequisites

- **Docker Desktop** (or Docker Engine)
- **Minikube**
- **kubectl**
- **Git**
- **Node.js** 20.19+ or 22.12+, with npm, only to run the API or the frontend outside containers. The bounds come from the MongoDB driver 7 (`>=20.19.0`) and Vite 8 (`^20.19.0 || >=22.12.0`).

6.2 Cluster Setup

The cluster runs in a dedicated Minikube profile, `incident-board`, which every `minikube` command must name; `kubectl` follows it on its own, since `minikube start` makes it the current context.

```
minikube start -p incident-board
minikube addons enable ingress -p incident-board
minikube addons enable metrics-server -p incident-board
kubectl create namespace app
```

The `ingress` addon installs the Nginx Ingress controller; `metrics-server` provides the CPU metrics without which the HPA's target stays `<unknown>` and `api` never scales.

6.3 Building the Container Images

Since the `api` and `web` manifests set `imagePullPolicy: Never`, both images (section 4.5) must be built inside Minikube rather than on the host. From the project root:

```
minikube image build -t api:latest ./api -p incident-board
minikube image build -t web:latest ./web -p incident-board
```

`mongo:6` and `redis:7` are official images, pulled by Kubernetes on first deployment.

6.4 Deploying the Kubernetes Manifests

From the project root, all the manifests are applied at once, then the pods are followed until they are `Running`:

```
kubectl apply -f k8s/ -n app
kubectl get pods -n app -w
```

Until the `mongo-rs-init` Job has initialized the replica set, the `api` pods exit at startup and are restarted by Kubernetes; they settle on their own once MongoDB is ready.

6.5 Reaching the Board

The Ingress matches the hostname `board.local`, which the host must resolve to an address where the Ingress controller listens.

Linux. The node's IP is routable from the host: add `<minikube-ip> board.local` to `/etc/hosts` (as administrator), with the output of `minikube ip -p incident-board`.

macOS with the `docker` driver. The node runs inside Docker Desktop's virtual machine and its IP is not routable. `minikube tunnel` bridges it: besides `LoadBalancer` Services, it forwards ports 80 and 443 to the Ingress controller through an SSH tunnel bound to the loopback address. It asks for the administrator password, because both ports are privileged, and its terminal must stay open:

```
echo "127.0.0.1 board.local" | sudo tee -a /etc/hosts
minikube tunnel -p incident-board
```

The board is then served at `http://board.local`.

7 User Guide

This section describes the application from the operator's point of view, assuming the deployment of section 6 is reachable at `http://board.local`.

7.1 Accessing the Board

A first-time operator clicks "Registrati" on the login card and chooses a username and a password of at least 8 characters; a successful registration logs the operator in directly. Later visits use the same credentials ("Accedi"). Either way the dashboard opens and the real-time connection is established in the background; wrong credentials are rejected with an inline error. The session survives a page reload and ends when the tab is closed, when the operator clicks "Esci" in the header, or after 12 hours. Both forms are shown in fig. 10.

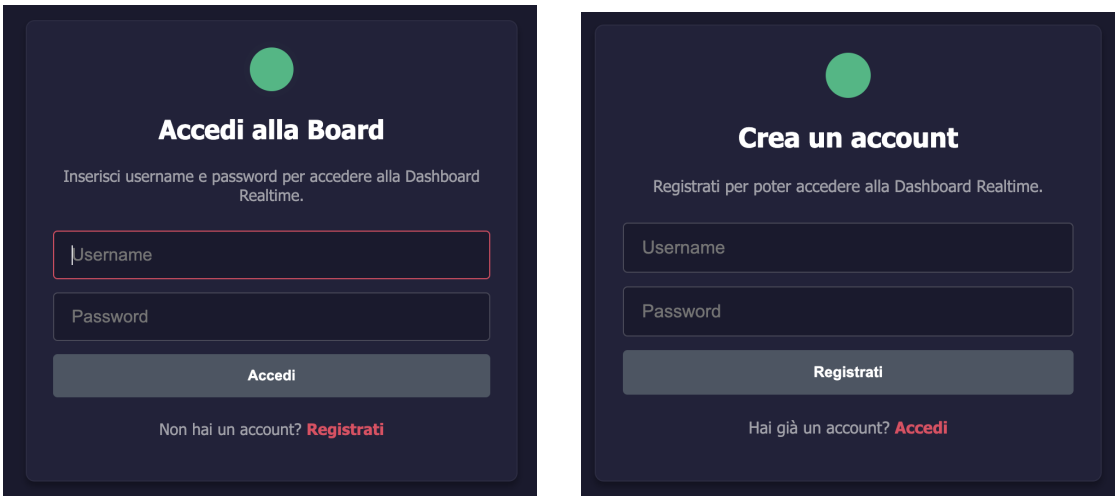


Figure 10: Login form (left) and registration form (right).

7.2 Dashboard Overview

The dashboard is organized in four rows, the first three of which are shown in fig. 11:

1. **KPI cards**: Total, Open, Escalated and Resolved tickets (by status, whether claimed or not) and Users Online, the number of distinct operators currently connected.

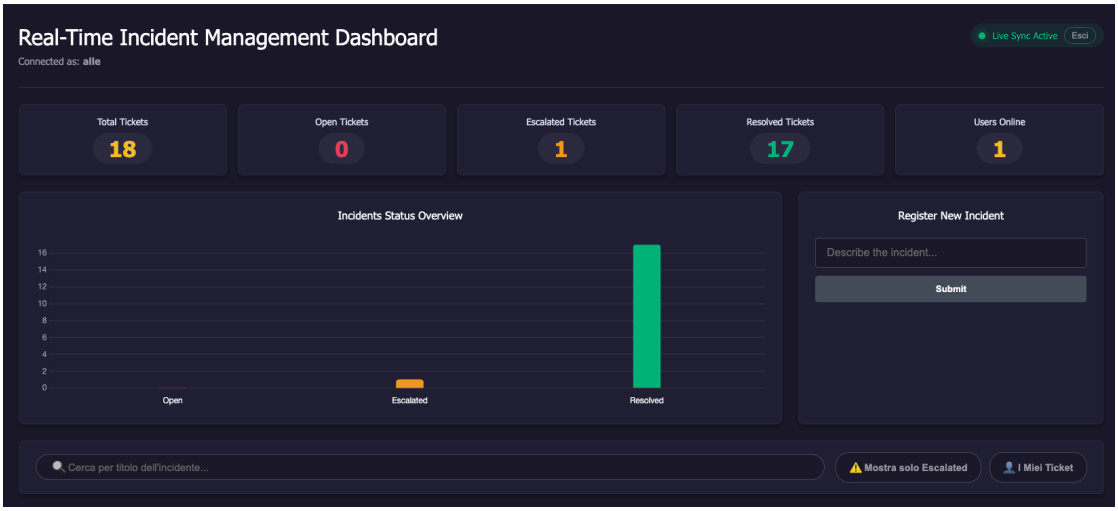


Figure 11: Upper part of the dashboard: header with the "Live Sync Active" label, KPI cards, status chart, creation form and filters.

2. **Status chart and creation form** ("Register New Incident").

3. **Filters**: a free-text search on the title (case-insensitive), "Mostra solo Escalated", and "I Miei Ticket", which keeps the incidents the operator created, holds, or resolved. Filters can be combined and affect *only* the Kanban columns: the KPI cards always describe the whole board.

4. **Kanban board** with four columns, each with a counter of its cards:
   - **Open** — unclaimed incidents with status `open`, with a "Claim Ticket" button;
   - **In Progress** — every claimed incident, whatever its status (an "ESCALATED" badge marks escalated ones), showing "Locked by: <name>"; only the holder sees the "Resolve" and "Release" buttons;
   - **Escalated** — unclaimed incidents past the SLA, with a "Claim Escalation" button;
   - **Resolved** — closed incidents, showing "Solved by: <name>", with no actions.

The "Live Sync Active" label in the header is static: it does not reflect the state of the real-time connection, which recovers on its own (section 7.5).

7.3 Incident Lifecycle

Create. Type a title in "Register New Incident" and press `Enter` or "Submit". The card appears in "Open" on every connected client, without any refresh.

Claim. Click "Claim Ticket" (or "Claim Escalation"). The card moves to "In Progress" on every client, and only the claimant gets the action buttons. If another operator got there first, an error toast reports that the incident is already claimed and nothing changes; the holder's name is already on the card. A claim lasts **60 seconds**: if it is neither resolved nor released by then, the card returns to its column on every client, including the holder's, and anyone can claim it again. Reloading the page does not lose a claim that is still valid.

Release. Click "Release" to give the incident back without resolving it. The card returns to "Open" or "Escalated" and is immediately claimable by others.

Resolve. Click "Resolve". The card moves to "Resolved" on every client, with the operator's name as resolver, and can no longer be claimed or modified. Only the current holder can resolve, and the server enforces this even for requests that bypass the interface. If the incident was escalated in the instant between the moment the card was last updated and the click, the request is rejected as a conflict ("Conflitto o errore: ..."); retrying once the card shows the new status succeeds, since the claim is still held.

Automatic escalation. An incident still `open` after **5 minutes** is escalated by the server, with no operator action: an unclaimed card moves to "Escalated", a claimed one stays in "In Progress" and gains the "ESCALATED" badge, and every client shows a warning toast. Escalated incidents follow the same claim/release/resolve workflow.

7.4 Notifications

Non-blocking toasts appear in the bottom-right corner and disappear after 5 seconds (8 for escalations); they can be dismissed by clicking or dragging them. They report the operator's own successful actions, incidents created or resolved by others (info and success), escalations (warning), and every failed action with the server's reason, such as a claim conflict or a network error (error).

7.5 Collaboration and Connection Resilience

The board is meant to be used by several operators at once: opening it in two browser windows with different users is enough to see cards move on both as either acts, the "Users Online" counter change as windows are opened and closed, and a claim on an incident held by the other user rejected immediately.

Connection problems require no action from the operator. If the connection drops (network glitch, pod restart, rolling update), the client reconnects by itself and reloads the whole board from the server, claims included, so it never shows a stale or partial state; the server can also ask its clients to reload when it detects that it has missed updates. Nothing is lost in the meantime: every action travels over HTTP and is either stored or reported as failed, while the real-time connection is used only to receive updates.

8 Self-evaluation

8.1 Role

The project was carried out individually, so every part of it is my own work: the elicitation of the requirements, the design of the architecture and of the coordination mechanisms, the implementation of the `api` backend and of the `web` frontend, the Kubernetes manifests, the manual validation on the Minikube cluster, and this report. Without a team there was also no reviewer: the coordination defects corrected during development were found by re-reading my own code against the properties it was meant to guarantee.

8.2 Strengths

- **One coordination primitive per problem.** Exclusive handling of a ticket, concurrent writes and the background task are three different problems, solved with three different tools — a Redis lock with atomic Lua scripts, optimistic locking on `version`, and leader election — each enforced by the server rather than trusted to the interface.
- **Real-time propagation without duplicates or polling.** Each event travels on exactly one channel: database changes through the per-pod Change Stream with `io.local.emit`, lock and presence events through the Redis adapter. Every client receives each event once, whichever pod it is connected to.
- **Recovery is designed, not assumed.** The Change Stream resumes from its token, its health drives the readiness probe, clients of a degraded pod are moved elsewhere, and every reconnection re-hydrates the whole board, claims included.
- **A genuinely stateless API tier.** Identity travels in a JWT and coordination state lives in Redis and MongoDB, so any request can land on any pod and the tier scales from 3 to 10 replicas without session migration.
- **Redundancy wherever it is cheap.** A 3-node MongoDB Replica Set, 3+ `api` and 2 `web` replicas, PodDisruptionBudgets, shallow/deep probes and a `preStop` hook on `web` keep the board available through pod crashes, rolling updates and the loss of a database node.

8.3 Weaknesses

- **Redis is a single point of failure.** It is the only component without redundancy, and its loss makes the whole board unavailable rather than read-only; having no persistence, a restart also drops every active claim.
- **Leader election is minimal.** Without a fencing token, a leader paused beyond the 15-second TTL can overlap with its successor, harmless only because escalation is idempotent. And since the lease lives in Redis alone, a leader cut off from

MongoDB keeps it while its scans fail, stopping escalation cluster-wide until the partition heals (section 3.8).

- **Validation is manual.** There is no automated test suite, so every regression check means re-running the scenarios by hand, and the checks added after the last validation round — the Change Stream as a readiness dependency among them — are documented as expected behaviour rather than observed (section 5.1).
- **Availability is pod-level, not node-level.** Minikube runs a single node: the replicas protect against crashes and rollouts, not against the loss of the machine.
- **Minimal security.** No TLS between components, a single role, no rate limiting on login, and a WebSocket session that outlives the expiry of its token (sections 3.9 and 9).
- **Scalability limits.** Each `api` pod holds its own Change Stream cursor on the primary, and every reconnection downloads the whole, unpaginated board.

9 Future works

The current iteration deliberately scopes out several directions in order to keep the focus on the distributed-systems aspects of the project. The most relevant candidates for future work are:

- **Transport security.** Traffic between the browser and the Ingress, and between the `api` pods and their backing services (MongoDB, Redis), is currently unencrypted. A production-grade deployment would terminate TLS at the Ingress and enable TLS for the MongoDB Replica Set and Redis connections, plus encryption at rest for the MongoDB volumes.
- **Fine-grained authorization.** All authenticated operators currently share a single implicit role. Introducing roles (e.g., operator vs. supervisor) with differentiated permissions — such as restricting who can resolve escalated incidents, or exposing an audit trail of who claimed/released/resolved which incident — would be a natural extension of the authorization model.
- **Account lifecycle hardening.** Login currently has no rate limiting, account lockout, or password-reset flow. Adding these (e.g., exponential backoff per username/IP, a lockout after repeated failures) would close an obvious brute-force gap without changing the stateless nature of the `api` tier.
- **Drag-and-drop interactions.** Kanban cards currently move between columns automatically as a side effect of status/lock changes; there is no direct manipulation. Adding drag-and-drop (e.g., dragging a card into "In Progress" to claim it) would need to reconcile the optimistic client-side move with the same claim/release/resolve endpoints and lock semantics already in place, rather than introduce a separate mutation path.